

Manipulation du DOM

Qu'est-ce que le DOM ?

- **DOM** = Document Object Model
- Représentation **arborescente** du document
- HTML Créé par le navigateur quand la page se charge
- JavaScript peut **lire, modifier, ajouter, supprimer** n'importe quel nœud

Qu'est-ce que le DOM ?

- HTML :

```
<p id="txt">Bonjour</p>
```

- DOM :

```
document
├── html
│   └── body
│       └── p (id="txt")
```

☑ Grâce au DOM, JavaScript peut :

- lire le contenu
- modifier le texte ou le style
- ajouter/supprimer des éléments
- réagir aux événements (clic, clavier...)

Fonctionnalité du Dom

1- Accéder
aux éléments
du DOM

2-Lire le
contenu

3-Lire et modifier
le contenu

5-Créer et ajouter
des éléments

4-Modifier le
style CSS

Accéder aux éléments du DOM

1-Par ID (le plus utilisé)

```
document.getElementById("txt");
```

2-Par classe

```
document.getElementsByClassName("box");
```

3- Par balise

```
document.getElementsByTagName("p");
```

4- Sélecteurs CSS

```
document.querySelector("p"); // premier
document.querySelectorAll(".box"); // tous
```

Exemple: Accéder aux éléments du DOM

```
<body>
  <!-- h1 -->
  <h1>Mon titre principal</h1>
  <h1>Deuxième titre</h1>
  <!-- paragraphes -->
  <p>Paragraphe 1</p>
  <p>Paragraphe 2</p>
  <!-- bouton avec ID -->
  <button id="monBouton">Clique ici</button>
  <!-- éléments avec classe -->
  <div class="item">Item 1</div>
  <div class="item">Item 2</div>
  <!-- liens -->
  <a href="#">Lien A</a>
  <a href="#">Lien B</a>
```

Exemple fichier html

```
// Méthodes les plus courantes
const titre = document.querySelector('h1');
// premier <h1> C'est un HTMLElement.

const paragraphes = document.querySelectorAll('p');
// tous les <p> Une NodeList (liste statique)

const btn = document.getElementById('monBouton');
// par ID, Objet unique (HTMLElement).

const items = document.getElementsByClassName('item');
// par classe (HTMLCollection)

const liens = document.getElementsByTagName('a');
// par balise (HTMLCollection)
```

Fichier .js

Exemple: Accéder aux éléments du DOM

Variable	Contenu précis	Type
titre	1 objet <h1>	HTML<Element>
paragraphes	Liste de <p>	NodeList
btn	1 bouton	HTML<Element>
items	Liste dynamique .item	HTMLCollection
liens	Liste dynamique <a>	HTMLCollection

Caractéristiques liste

C'est une **liste d'éléments**.
Elle est généralement **statique** (ne change pas si le DOM change).
On peut utiliser directement

Caractéristiques collection

C'est une **collection vivante** (live).
Elle se met à jour automatiquement si on ajoute/supprime des éléments.
✗ Pas de forEach() direct.

Lire le contenu

Propriété / Méthode	Ce qu'elle renvoie
<code>element.textContent</code>	Le texte pur (sans balises)
<code>element.innerHTML</code>	Le HTML interne (balises incluses)
<code>element.innerText</code>	Texte visible (respecte le CSS)
<code>element.value</code>	Valeur d'un input, textarea, select

Lire le contenu (textContent)

- Lire le contenu texte:

```
console.log(titre.textContent); //Affiche: Mon titre principal
```

- Lire le texte des paragraphes (NodeList)

```
paragraphes.forEach(p => {  
  console.log(p.textContent);  
}); /* Affiche: Paragraphe 1  
      Paragraphe 2 */
```

- Lire le texte du bouton

```
console.log(btn.textContent); // Affiche Cliquez ici
```

Lire le contenu (textContent)

- Lire le contenu des items (HTMLCollection)

```
for(let i = 0; i < items.length; i++){  
  console.log(items[i].textContent);  
}  
/* Affiche Item 1  
   Item 2 */
```

Lire le contenu

Propriété / Méthode	Ce qu'elle renvoie	Exemple
<code>element.innerHTML</code>	Le HTML interne (balises incluses)	titre. <code>innerHTML</code> → <code><h1>Mon titre principale </h1></code>
<code>element.textContent</code>	Le texte pur (sans balises)	titre. <code>textContent</code> → Mon titre principal
<code>element.innerText</code>	Texte visible (respecte le CSS)	titre. <code>innerText</code> → Mon titre principal
<code>element.value</code>	Valeur d'un input, textarea, select	input. <code>value</code> → ce que l'utilisateur a tapé

Lire et modifier le contenu

1-Texte

```
element.textContent = "Nouveau texte";
element.innerText = "Visible seulement";
```

2-HTML

```
element.innerHTML = "<strong>Texte</strong>";
```

Modifier le style CSS

1-Style direct

```
element.style.color = "red";
element.style.fontSize = "20px";
```

2-Classes CSS

```
element.classList.add("active"); // ajout de la classe active a l'élément
element.classList.remove("active");// supprime la classe active de l'élément
element.classList.toggle("active"); /*Si l'élément n'a pas la classe "active", elle
est ajoutée. Si l'élément a déjà la classe "active", elle est retirée. Exemple
Boutons on/off */
```

Remarque: Toujours ajoutée, même si l'élément a déjà d'autres classes.

Ne fait pas de doublon : si l'élément a déjà la classe "active", elle ne sera pas ajoutée une 2^e fois.

Créer et ajouter des éléments

• Étape 1 : Créer un élément

```
document.createElement("nomDeLaBalise")
```

Exemple :

```
const nouveauParagraphe = document.createElement("p");
```

• Étape 2 : Ajouter du contenu

On peut ajouter du texte avec `textContent` ou du HTML avec `innerHTML` :

```
nouveauParagraphe.textContent = "Je suis un nouveau paragraphe";
```

Créer et ajouter des éléments

•Étape 3 : Ajouter l'élément dans le DOM

Pour l'ajouter dans la page, on utilise par exemple :

appendChild → ajoute à la fin
prepend → ajoute au début
insertBefore → ajoute avant un élément précis

Exemple:

```
const container = document.querySelector("body");
container.appendChild(nouveauParagraphe);
```

Maintenant le paragraphe est **visible dans la page** à la fin du body.

Manipuler les attributs HTML

- Lire un attribut : `getAttribute("nom")`
- Modifier / ajouter un attribut : `setAttribute("nom", "valeur")`
- Supprimer un attribut : `removeAttribute("nom")`

Exemple

Fichier html

```

<button id="btn" disabled>Cliquer</button>
```

Lire modifier un attribut

```
const img = document.getElementById("photo");
console.log(img.getAttribute("src"));
img.setAttribute("src", "photo2.png");
```

Supprimer un attribut

```
const btn = document.getElementById("btn");
btn.removeAttribute("disabled");// bouton devient cliquable
```


Les Événements

- Un **événement** est une action de l'utilisateur ou du navigateur (clic, saisie, chargement).
- JavaScript peut **écouter** et **réagir** à ces événements.
- Grâce au JavaScript il est possible d'associer des fonctions, des méthodes à des événements tels que le passage de la souris au-dessus d'une zone, le changement d'une valeur,

Événements courants:

- **click** : clic sur un élément.
- **mouseover** : passage de la souris.
- **submit** : envoi d'un formulaire.
- **Saisie** dans un input (input, change)
- **Chargement** de la page (load)

Les Événements

- Syntaxe générale:

```
element.addEventListener("nomEvenement", fonction, useCapture);
```

Les Événements

- Les types d'événements les plus courants

Type	Description	Exemple
click	Clic souris	<button>
dblclick	Double-clic	<div>
mouseover / mouseout	Souris sur / hors élément	<div>
keydown / keyup	Touche du clavier	<input>
input	Texte saisi	<input>
change	Valeur modifiée	<select>
submit	Formulaire soumis	<form>
load	Page / image chargée	<window> /
Scroll	Défilement	<window>

Obtenir l'élément qui a déclenché l'événement

- Obtenir l'élément qui a déclenché l'événement:
- Dans `addEventListener`, la fonction reçoit un **objet event** :

```
btn.addEventListener("click", function(event) {
  console.log(event.target); // élément cliqué
});
```

Obtenir l'élément qui a déclenché l'événement:

`event.preventDefault()` → empêche le comportement par défaut (ex : soumission d'un formulaire).

`event.stopPropagation()` → empêche la propagation de l'événement aux parents (bubble).

`event.target` → élément exact qui a déclenché l'événement.

Exemples pratiques

A. Clic sur un bouton pour changer le texte

```
btn.addEventListener("click", () => {  
  btn.textContent = "Merci !";  
});
```

B. Survol pour changer la couleur

```
const div = document.querySelector(".item");  
div.addEventListener("mouseover", () => {  
  div.style.backgroundColor = "yellow";  
});
```

C. Formulaire → empêcher l'envoi

```
const form = document.querySelector("form");  
form.addEventListener("submit", (e) => {  
  e.preventDefault();  
  alert("Formulaire bloqué !");  
});
```